

URL SCANNING AND VIRUS DETECTION SYSTEM USING N8N AUTOMATION

*A Capstone Project-2 report Submitted in partial fulfillment of the requirements
for the award of degree of*

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

SADIQUDDIN MAHAMMAD

2200032785

GADEPALLI VARUN

2200030746

GURRAM SRINIVAS

2200032417

Under the supervision of

Dr. M. LAKSHMANA PHANEENDRA

ASSOCIATE PROFESSOR



Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
(Deemed to be University estd., u/s 3 of UGC Act 1956)
Green Fields, Vaddeswaram-522502, Guntur (Dist.), Andhra Pradesh, India
APRIL 2026

KONERU LAKSHMAIAH EDUCATION FOUNDATION

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Declaration

The Capstone Project-2 Report entitled "**URL SCANNING AND VIRUS DETECTION SYSTEM USING N8N AUTOMATION**" is a record of Bonafide work of **SADIQUDDIN MAHAMMAD (2200032785), GURRAM SRINIVAS (2200032417), GADEPALLI VARUN (2200030746)** submitted in partial fulfillment for the award of Bachelor of Technology in Computer Science and Engineering to K L Deemed to be University during the academic year 2025-26.

We also declare that this report is of our own effort, and it has not been submitted to any other university for the award of any degree.

SADIQUDDIN MAHAMMAD	2200032785
GADEPALLI VARUN	2200030746
GURRAM SRINIVAS	2200032417

KONERU LAKSHMAIAH EDUCATION FOUNDATION

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Certificate

This is to certify that the Capstone Project-2 Report entitled “**URL SCANNING AND VIRUS DETECTION SYSTEM USING N8N AUTOMATION**” is being submitted by **SADIQUDDIN MAHAMMAD (2200032785), GURRAM SRINIVAS (2200032417), GADEPALLI VARUN (2200030746)** in partial fulfillment for the award of Bachelor of Technology in Computer Science and Engineering to K L Deemed to be University during the academic year 2025-26.

Signature of the Supervisor

Dr. M. LAKSHMANA PHANEENDRA
ASSOCIATE PROFESSOR

Capstone Project-2 Coordinator

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENTS

It is great pleasure for me to express my gratitude to our Hon'ble Chancellor **Sri. Koneru Satyanarayana**, for giving the opportunity and platform with facilities in accomplishing the capstone project-1 course successfully.

I express sincere gratitude to our principal **Dr. V. RAJESH** for his administration towards our academic growth.

I record it as my privilege to deeply thank our pioneer's **Prof. V. Hari Kiran**, Dean (Addl) Academics and **Dr. T. Pavan Kumar**, HOD, CSE-4 Department for providing us the efficient faculty and facilities to make our ideas into reality.

We express our sincere thanks to our project supervisor **Dr. M. Lakshmana Phaneendra**, Associate Professor for his novel association of ideas, encouragement, appreciation, and intellectual zeal which motivated us to Complete this capstone project-2 work successfully.

Finally, it is pleased to acknowledge the indebtedness to all those who devoted directly or indirectly to making this the capstone project-2 success.

Capstone Project-2 Associate

Name	Student ID
SADIQUDDIN MAHAMMAD	2200032785
GADEPALLI VARUN	2200030746
GURRAM SRINIVAS	2200032417

ABSTRACT

In today's digital environment, malicious URLs have become one of the most widely used vectors for phishing, malware distribution, credential theft, and other cyberattacks. Traditional rule-based filters and signature-matching systems provide only limited protection, as they fail to detect newly generated or rapidly evolving malicious URLs. This highlights the need for an adaptive, intelligent, and automated detection mechanism capable of identifying threats in real time.

This project proposes a Hybrid Malicious URL Detection System that combines n8n-based workflow automation with a custom Python detection engine. Instead of relying on external APIs, it employs four internally implemented algorithms—Random Forest, Logistic Regression, Lexical Analysis, and Blacklisting—to classify URLs. All four methods evaluate each URL, and their combined decisions produce the final verdict, improving accuracy through a multi-model approach.

The detection engine is exposed through a lightweight Flask API, enabling seamless integration with n8n workflows. The n8n platform handles URL intake, sends requests to the Python server, evaluates the results from each algorithm, and automatically sends an email alert highlighting which algorithms flagged the URL as malicious. This automation reduces human effort, speeds up threat identification, and ensures consistent and repeatable analysis.

Key Points:

- To create an automated system that detects malicious URLs and files using n8n.
- Cyber Threats are increasing in frequency and complexity.
- n8n: For workflow automation.
- Real-time URL scanning.
- Flask API enables communication between n8n and the detection engine.
- Python engine applies four algorithms: RF, LR, Lexical, and Blacklisting.

This system offers a practical and scalable approach to malicious URL detection through a blend of machine learning, heuristics, and automated workflows.

LIST OF FIGURES

Figure Number	Figure Name	Page No
3.1	Automation workflow using send alert via Gmail notification	17
3.4	Block Diagram of Random Forest Algorithm for URL Classification	20
3.5	Heuristic URL Analysis Algorithm Based on Rule-Based Risk Scoring	22
3.6	URL Blacklisting Heuristic Workflow for Domain Reputation Checking	23
4.2	End-to-End Logistic Regression Model Workflow for URL Classification	27
4.3	n8n Workflow for Automated Malicious URL Detection	28
5.2	Automated Gmail Alert Generated When the Random Forest Model Detects a Malicious URL	34

LIST OF TABLES

Table Number	Table Name	Page No
3.1	Comparison of Algorithms Used in the Malicious URL Detection System	21
5.1	Accuracy of the Four Algorithms Used for URL Classification	32

TABLE OF CONTENTS

Chapter Number	Title	Page Number
1	Chapter 1: Introduction	9-11
2	Chapter 2: Literature Survey	12-14
3	Chapter 3: Design and Methodology	15-25
4	Chapter 4: Experimental Investigation	26-30
5	Chapter 5: Expected Results and Discussion	31-41
6	Chapter 6: Conclusion & Future Work	42-46
7	Chapter 7: References	47-50

Chapter 1: INTRODUCTION

1.1 Background of study

In the modern digital landscape, malicious URLs have become a major vector for phishing, malware distribution, and unauthorized access. These threats are often disguised to appear legitimate, making them difficult to detect through traditional rule-based or manual methods. Attackers frequently generate dynamic and short-lived URLs, making reactive approaches insufficient for timely protection. As a result, organizations require more adaptive and efficient techniques to identify harmful links before they cause data breaches, financial loss, or service disruption.

To address this need, automation and analytical detection methods have gained importance in cybersecurity workflows. Tools like **n8n** provide a flexible platform to automate tasks such as URL intake, verification, and alerting. However, effective detection also requires intelligent analysis capable of identifying suspicious URL characteristics. This project adopts a hybrid strategy by integrating **machine learning and heuristic techniques** within a Python-based detection engine. Using four analytical methods—**Random Forest, Logistic Regression, Lexical Analysis, and Blacklisting**—the system evaluates URLs from multiple perspectives and enables reliable real-time classification. The combination of automated processing and multi-algorithm detection enhances accuracy and improves overall threat response.

1.2 Motivation

The motivation for this project comes from the increasing difficulty cybersecurity teams face in detecting malicious URLs quickly and accurately. As phishing links and harmful websites continue to grow in number and complexity, traditional manual inspection and rule-based filtering methods are no longer sufficient to ensure timely protection. Security analysts often deal with large volumes of alerts, making it challenging to respond effectively to every potential threat. To address these issues, this project aims to create an automated and reliable detection system that can analyze URLs from multiple perspectives using a combination of machine learning and heuristic techniques. By integrating n8n for workflow automation with a Python-based multi-algorithm

detection engine, the project seeks to reduce manual effort, improve detection consistency, and support faster decision-making in modern cybersecurity operations.

1.3 Problem Statement

Although many tools exist for detecting malicious URLs, organizations still face difficulties in creating an automated, unified, and reliable workflow for real-time analysis. Most existing systems depend heavily on manual inspection or single-method detection techniques, which leads to slow response times and inconsistent results, especially when dealing with newly generated or obfuscated threats. Furthermore, integration challenges between detection engines and operational workflows often limit the efficiency of security teams. There is a need for a system that can automatically evaluate URLs using multiple analytical methods, reduce reliance on manual effort, and provide faster, more accurate threat classification. This project addresses this gap by developing a hybrid detection framework that combines workflow automation through n8n with a Python-based multi-algorithm engine capable of identifying malicious URLs more effectively.

1.4 Objectives of the Study

- To design and implement an automated malicious URL detection system using n8n.
- To develop a Python-based detection engine that applies four analytical methods: Random Forest, Logistic Regression, Lexical Analysis, and Blacklisting.
- To enable real-time URL evaluation through a Flask API integrated with automated workflows.
- To reduce manual effort by automating URL intake, analysis, and alert generation.
- To assess the system's accuracy and reliability through multi-algorithm comparison and performance evaluation.

1.5 Scope of the Project

The scope of this project includes developing a prototype cybersecurity solution using n8n that automates the process of URL scanning, virus detection, and alert generation. The system is intended for small to mid-sized organizational environments and focuses on integrating open-source tools and publicly available threat intelligence services. While the project demonstrates the feasibility of automated and intelligent threat detection, it does not cover large-scale deployment, advanced malware analysis, or compliance-related functionalities.

Chapter 2: Literature Survey

2.1 Introduction

The rapid rise of web-based attacks has made malicious URL detection an essential focus in cybersecurity research. Threat actors increasingly rely on deceptive, short-lived, and obfuscated URLs to spread malware, steal credentials, and mislead users, making traditional rule-based filters and static blacklists insufficient for timely protection. These limitations have encouraged the development of adaptive analytical methods capable of identifying malicious patterns more effectively.

Recent studies show that using multiple detection approaches leads to more reliable results. Machine learning models like Random Forest and Logistic Regression identify structural patterns in URLs, while heuristic methods such as lexical analysis and blacklist checks quickly flag known indicators and high-risk characteristics. Combining these techniques in a single system improves accuracy and strengthens defence against different types of the malicious URLs. Automation platforms like n8n further support this process by handling URL intake, analysis flow, and alert generation automatically, ensuring faster and more consistent decision-making without manual effort.

This section reviews relevant research on malicious URL detection, emphasizing machine learning-based techniques, heuristic approaches, and automation-driven workflows. These studies collectively form the foundation for the hybrid multi-algorithm detection framework developed in this project.

2.2 Heuristic-Based Detection Techniques

Heuristic-based techniques contribute significantly to malicious URL detection by evaluating predefined characteristics and structural patterns often found in harmful links. Unlike machine learning models that require training data, heuristic methods use rule-driven checks such as URL length, special character count, suspicious keywords, domain patterns, and blacklist reputation.

These methods are effective for identifying known malicious sources and URLs that show obvious signs of abnormal behaviour. When integrated with automation tools like n8n, heuristic analysis can run automatically during URL intake, generating immediate alerts and classification results. This makes heuristics a lightweight, fast, and interpretable component within modern URL detection systems.

2.3 Integration Challenges

Integrating multiple detection techniques into a unified malicious URL analysis framework introduces a variety of technical, architectural, and operational challenges that must be carefully managed for the system to perform reliably. Workflow automation tools such as n8n need to communicate seamlessly with the external Python-based detection engine, which requires stable API endpoints, consistent request/response formatting, and well-designed error-handling mechanisms. Even minor inconsistencies in JSON structures, HTTP headers, or response codes can interrupt the entire pipeline, forcing workflows to fail or repeat tasks unnecessarily.

Another significant challenge lies in coordinating the different processing speeds and computational demands of the four chosen algorithms—Random Forest, Logistic Regression, Lexical Analysis, and Blacklisting. Each method operates on different principles and generates outputs in unique formats. Merging these diverse results into a single unified verdict requires careful design of aggregation logic, confidence scoring, and tie-breaking rules. Additionally, handling concurrency becomes crucial when the system processes many URLs in rapid succession, as the backend must respond quickly without timing out or overloading the automation workflow. Maintaining long-term model performance introduces further complexity. Machine learning models must be retrained periodically with updated datasets to ensure they can detect emerging patterns, while heuristic rules and blacklist sources must be refreshed regularly to stay relevant against evolving threat indicators. Variations in URL structure—such as dynamically generated strings, encoded parameters, or shortened links—also pose difficulties, requiring more advanced preprocessing and normalization to ensure the models receive clean and meaningful inputs.

Finally, achieving consistent integration across all components demands a continuous balance between accuracy, speed, and reliability. Any disruptions—such as API rate limits, server delays,

missing data fields, or inconsistencies across algorithms—can negatively impact detection quality. Therefore, addressing these integration challenges is essential for creating a fast, dependable, and scalable automated URL detection framework capable of functioning effectively in real-world environments.

2.4 Research Gaps and Opportunities

Although significant progress has been made in malicious URL detection, existing approaches still face limitations that create several research opportunities. Many systems rely on a single analytical technique, making them less effective against newly generated or evolving threats. Traditional blacklist-based or rule-based methods are fast but often inaccurate, while machine learning models perform better but require ongoing maintenance and retraining. Literature also highlights a lack of integrated frameworks that combine multiple detection methods within a lightweight, automated workflow. Open-source tools like n8n provide flexibility, but their use in security-focused URL analysis is still underexplored. This creates opportunities to develop hybrid detection systems that unify machine learning, heuristic analysis, and workflow automation to enhance accuracy, scalability, and real-time decision-making.

Chapter 3: Design and Methodology

The system integrates n8n workflow automation with a Python-based detection engine to provide efficient and reliable malicious URL analysis. n8n manages URL intake, backend communication, and automated alerting, while the Python engine evaluates each URL using four techniques: Random Forest, Logistic Regression, Lexical Analysis, and Blacklisting. The machine learning models identify patterns learned from training data, whereas heuristic methods detect structural anomalies and known malicious domains.

All four detection techniques operate in parallel, and their outputs are combined using a unified scoring mechanism. Each method contributes equally to the overall malicious score, ensuring balanced decision-making. The final verdict is determined based on this aggregated score, where a threshold is applied to classify the URL as malicious or benign.

Based on the final classification returned by the backend, n8n automatically triggers an alert email containing detailed analysis results. This integrated design ensures fast processing, consistent detection, and timely notifications within a lightweight, scalable, and modular framework.

3.1 System Architecture Overview

The proposed system is designed as a hybrid malicious URL detection framework that integrates workflow automation with multi-algorithm analysis to achieve fast, reliable, and scalable threat identification. The architecture consists of two major components: the n8n automation workflow, which manages URL intake, processing, and alerting, and the Python-based detection engine, which performs the actual classification using multiple analytical methods. These components operate collaboratively to ensure that every URL is evaluated efficiently and consistently.

At the entry point of the system, n8n acts as the workflow controller, receiving URL inputs through a webhook and forwarding them to the backend detection engine. The detection engine, implemented using Python and exposed through a lightweight Flask API, processes each URL through four different techniques—Random Forest, Logistic Regression, Lexical Analysis, and

Blacklisting. Each method examines the URL from a distinct perspective: machine learning models analyse learned patterns, while heuristic techniques evaluate structural characteristics and known malicious sources.

All four detection mechanisms operate in parallel to improve efficiency and reduce processing time. Each algorithm independently produces a Boolean output indicating whether the URL is malicious or not. These outputs are then collected and merged within the backend system, where a centralized scoring mechanism is applied. Each algorithm contributes equally (25%) to the overall malicious score, ensuring a balanced evaluation across all techniques.

The final verdict is determined based on the aggregated malicious score. If the score is greater than or equal to 50%, the URL is classified as malicious; otherwise, it is considered benign. This approach ensures that the system does not rely on a single detection method, thereby improving robustness and reducing false positives or negatives.

Once the backend returns the classification results in JSON format, n8n evaluates the outputs using conditional logic. Based on the final verdict, the system automatically triggers an appropriate action, such as sending a detailed alert email. The email includes a structured, line-by-line report containing the URL, final verdict, malicious score, and the specific detection methods that identified the URL as malicious.

This architecture ensures a smooth flow of data, minimal manual intervention, and a modular design in which each component can be updated or extended independently. By blending automation, machine learning, and heuristic detection, the system offers a robust, scalable, and adaptable approach to identifying harmful URLs in real time.

- Integrates automation (n8n) with multi-algorithm detection for real-time processing
- Executes four detection techniques in parallel for improved efficiency
- Uses equal-weight scoring (25% per algorithm) for balanced decision-making
- Applies a threshold-based verdict system ($\geq 50\%$ = Malicious)
- Generates automated, detailed email alerts with detection insights

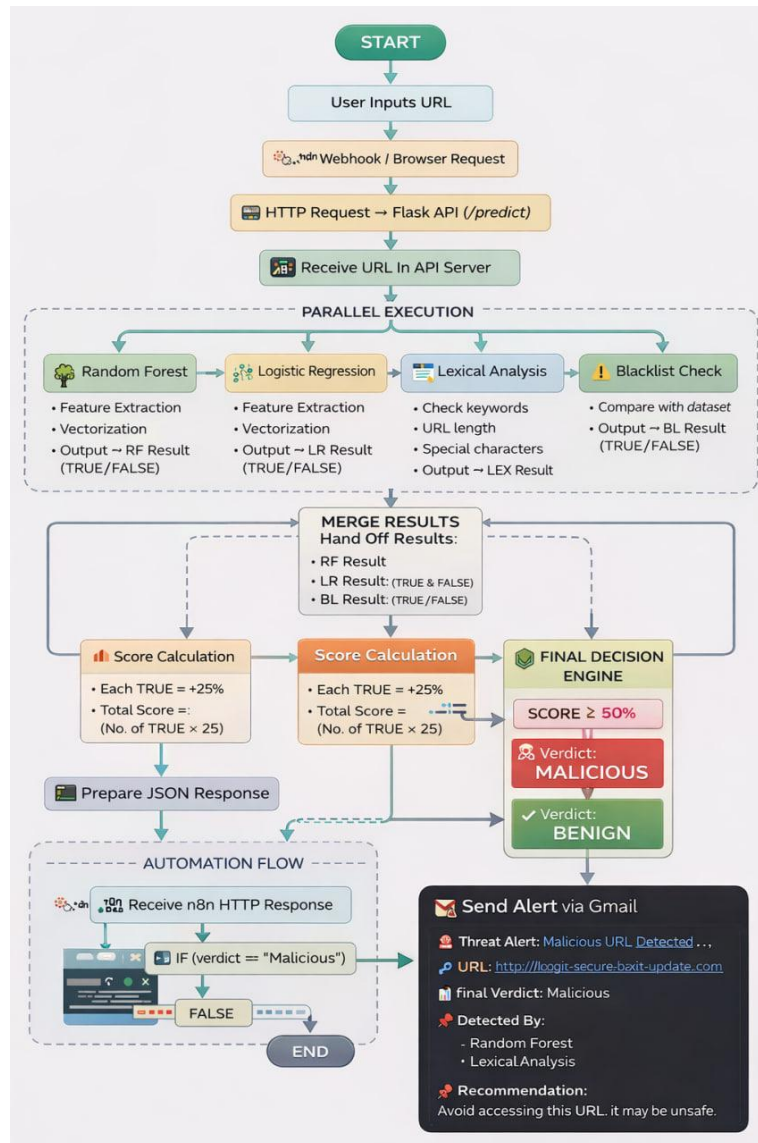


Figure 3.1: Automation workflow using send alert via Gmail notification

3.2 Workflow Automation Using n8n

The n8n platform functions as the automation layer of the system, enabling seamless coordination of URL intake, backend communication, and alert generation. It provides a structured workflow in which each step is executed automatically without requiring manual intervention. The process begins with a Webhook node, which receives the URL submitted by the user or an external application. Once the input is captured, the workflow forwards the URL to the backend detection engine using an HTTP Request node.

The HTTP Request node communicates with the Python-based API, sending the URL for processing and receiving the classification results in JSON format. This response includes the outputs from all four detection techniques along with the final verdict and malicious score. n8n then evaluates the response using an IF node, which checks whether the final verdict is classified as malicious.

If the URL is identified as malicious, the workflow triggers an alert through the Gmail node. The generated email contains a detailed, structured report including the URL, final verdict, malicious score, and the detection methods that flagged the URL. If the URL is benign, no further action is taken, ensuring efficient workflow execution.

The use of n8n simplifies automation and enhances flexibility, allowing additional processing steps or integrations to be added easily in the future. Overall, it provides a reliable and scalable pipeline that reduces manual effort while ensuring consistent and real-time URL analysis.

3.3 Python-Based Detection Engine

The Python-based detection engine serves as the core analytical component of the system, responsible for evaluating each submitted URL using a combination of machine learning and heuristic techniques. The engine is implemented using Python due to its strong support for data processing, feature extraction, and model deployment. It is exposed through a lightweight Flask API, which receives incoming URL requests, processes them, and returns classification results in a structured JSON format.

Upon receiving a URL, the engine extracts lexical and structural features such as URL length, character composition, presence of suspicious keywords, and domain patterns. These features are utilized by the machine learning models—Random Forest and Logistic Regression—which are trained on a labelled dataset to distinguish between benign and malicious URLs.

In parallel, the engine performs heuristic analysis using Lexical Analysis rules and a Blacklist check to identify known malicious domains. Each of the four techniques produces an independent boolean output, indicating whether the URL is considered malicious by that method.

The outputs from all detection techniques are then aggregated within the system. A scoring mechanism assigns equal weight (25%) to each method, and the total malicious score is calculated based on the number of positive detections. The final verdict is determined using a threshold-based approach, where a score of 50% or higher classifies the URL as malicious.

The engine returns a consolidated JSON response containing the URL, individual algorithm results, malicious score, and final verdict. This modular and extensible design allows additional detection techniques to be integrated easily, ensuring scalability and adaptability for real-time security applications.

3.4 Machine Learning Model Development

The machine learning component of the system is designed to classify URLs as malicious or benign based on patterns learned from labeled data. Two supervised learning algorithms—Random Forest and Logistic Regression—are implemented to ensure both accuracy and efficiency in detection. These models are trained using the `malicious_phish.csv` dataset, which contains labeled examples of various types of URLs.

Random Forest is an ensemble-based classification algorithm that builds multiple decision trees and combines their outputs to produce a final prediction. Each tree analyses different aspects of the input features, such as URL length, character distribution, and keyword presence. The final classification is determined through majority voting among all trees, which improves accuracy and reduces overfitting.

In this system, Random Forest is particularly effective in capturing complex and non-linear patterns present in malicious URLs. Its ability to handle diverse feature sets makes it a strong component of the detection framework.

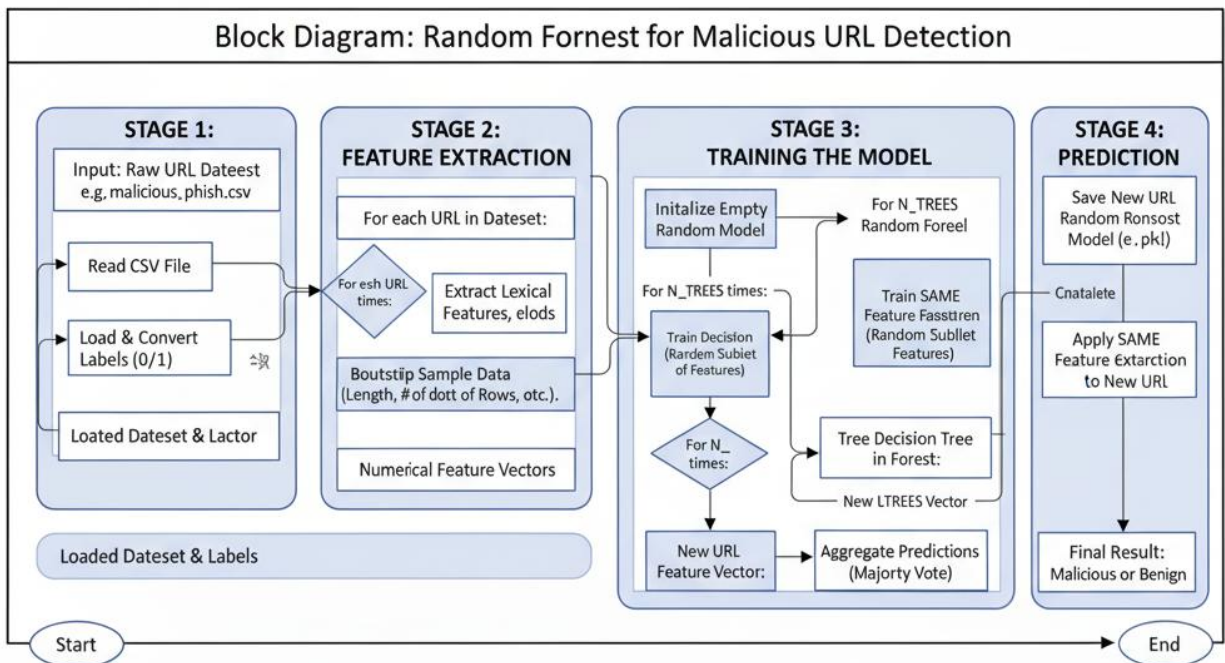


Figure 3.4: Block Diagram of Random Forest Algorithm for URL Classification

Within this system, Logistic Regression provides a lightweight and fast prediction mechanism. It is especially useful for capturing linear relationships between features and serves as a complementary model to Random Forest, ensuring balanced detection performance.

Before applying the machine learning models, URLs are processed to extract meaningful lexical and structural features. These include URL length, number of digits, presence of special characters, and suspicious keywords. The extracted features are then converted into numerical vectors using a vectorization technique.

The trained models are saved and loaded during runtime, allowing real-time prediction without retraining. Each model produces a boolean output indicating whether the URL is malicious. These outputs are later combined with heuristic results to contribute to the overall malicious score.

By integrating both Random Forest and Logistic Regression, the system benefits from the strengths of ensemble learning and probabilistic modeling, resulting in improved detection accuracy and robustness.

Comparison Point	Random Forest (ML)	Logistic Regression (ML)	Lexical Analysis (Heuristic)	Blacklisting (Heuristic)
1. Detection Method	Learned (ML): Learns complex patterns from a large dataset.	Learned (ML): Learns statistical probabilities from a large dataset.	Rule-Based: Calculates a risk score based on pre-programmed rules about the URL string.	Rule-Based: Checks if the URL is present in a pre-defined list of known threats.
2. Dataset Requirements	High: Requires a large, labeled dataset for a slow training process	High: Requires a large, labeled dataset for a fast-training process	None: Does not require a dataset to function, only rules.	High: Requires a massive, constantly updated database of known malicious URLs
3. Speed & Performance	Slow to Train, Fast to Predict.	Fast to Train, Very Fast to Predict.	Instant (Very Fast): No training needed, prediction is a simple string analysis.	Instant (Very Fast): No training needed, prediction is a simple list

Table 3.1: Comparison of Algorithms Used in the Malicious URL Detection System

3.5 Heuristic Techniques (Lexical Analysis and Blacklisting)

Heuristic techniques play a crucial role in the system by providing fast and interpretable rule-based evaluation of URLs. Unlike machine learning models, these methods do not require training and are effective in identifying obvious indicators of malicious behavior. Two heuristic approaches are implemented: Lexical Analysis and Blacklisting.

Lexical Analysis evaluates the structure and composition of a URL to detect suspicious patterns. It examines features such as URL length, ratio of digits to letters, number of special characters, and the presence of high-risk keywords like “login,” “verify,” “update,” and “secure.” Additionally, URLs with excessive subdomains, hyphens, or unusual formatting are flagged as potentially malicious.

This method provides quick and explainable results, making it useful for identifying suspicious URLs without complex computation.

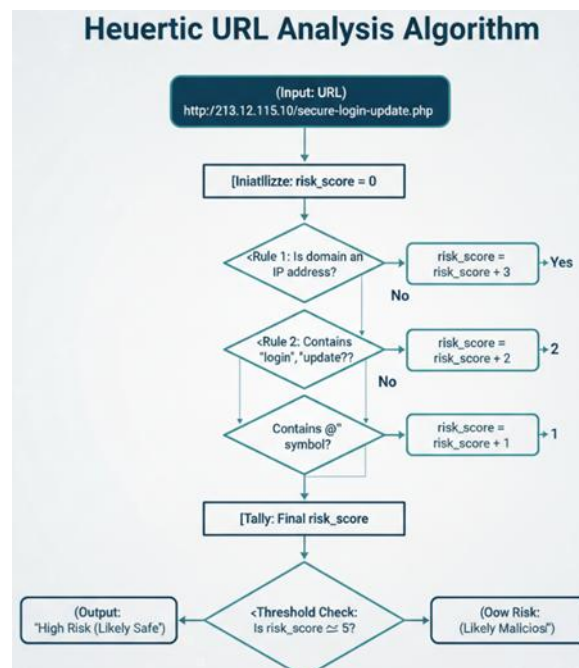


Figure 3.5: Heuristic URL Analysis Algorithm Based on Rule-Based Risk Scoring

Blacklisting is a rule-based technique that checks whether a given URL exists in a predefined list of known malicious domains or addresses. The system uses a dataset of previously identified malicious URLs for comparison. If a match is found, the URL is immediately classified as malicious.

While highly accurate for known threats, blacklisting cannot detect new or unseen malicious URLs. Therefore, it is used in combination with machine learning techniques to improve overall detection coverage.

Both heuristic methods produce boolean outputs indicating whether a URL is suspicious. These results are integrated with the outputs of machine learning models. Each detection method contributes equally to the overall malicious score, ensuring a balanced and reliable classification. By combining lexical analysis and blacklisting with machine learning techniques, the system achieves faster detection, improved interpretability, and enhanced robustness against both known and unknown threats.

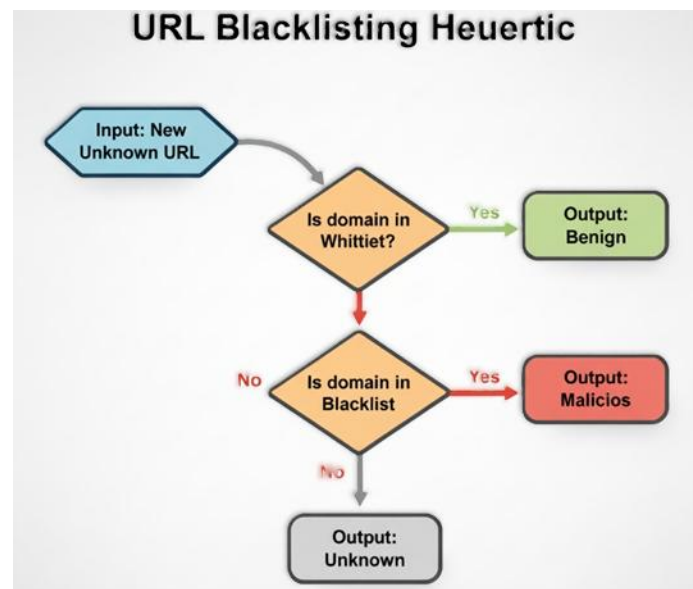


Figure 3.6: URL Blacklisting Heuristic Workflow for Domain Reputation Checking

3.6 Real-Time Dashboard Updates

The final stage of the system focuses on combining the outputs of all detection techniques to produce a unified and reliable classification for each URL. The system employs four analytical methods—Random Forest, Logistic Regression, Lexical Analysis, and Blacklisting—each generating an independent Boolean result indicating whether the URL is malicious.

These outputs are aggregated using a scoring-based decision mechanism. Each detection method contributes equally (25%) to the overall malicious score, ensuring a balanced evaluation across all techniques. The total score is calculated based on the number of methods that classify the URL as malicious, preventing any single method from dominating the final decision.

The final verdict is determined using a threshold-based rule. If the malicious score is greater than or equal to 50%, the URL is classified as malicious; otherwise, it is considered benign. This approach provides a consistent and interpretable decision-making process.

Once the detection engine computes the results, it returns a structured JSON response to the n8n workflow. The workflow evaluates the response using an IF node, which checks the final verdict. If the URL is classified as malicious, n8n automatically triggers an alert through the email notification system.

The generated alert includes detailed information such as:

- URL submitted for analysis
- Final classification (Malicious/Benign)
- Calculated malicious score (%)
- Detection methods that identified the URL as malicious
- Multi-algorithm result aggregation for improved accuracy
- Equal-weight scoring system (25% per detection method)
- Threshold-based classification ($\geq 50\%$ indicates malicious)
- Automated decision-making using n8n IF node
- Real-time email alerts with detailed analysis
- Clear and explainable output for user interpretation

By integrating multiple detection techniques with automated alerting, this stage ensures efficient, accurate, and real-time response to potential threats while maintaining a scalable and modular system design.

Chapter 4: Experimental Investigation

To rigorously assess the performance, scalability, and real-world applicability of the proposed automated cybersecurity framework, an experimental setup was meticulously designed to simulate the operational environment of a small-to-medium-sized enterprise Security Operations Centre (SOC). The primary objective of this phase was to validate the effectiveness of integrating automation, deception, and AI-driven analytics within a unified threat detection and response architecture.

4.1 Infrastructure Setup

The infrastructure for the proposed system is built to ensure smooth interaction between the automation workflow and the multi-algorithm detection engine. It consists of two main components operating in a lightweight environment: the **n8n workflow automation platform** and the **Python-based backend server**. Both components can run on a standard system, making the setup simple and suitable for small to mid-sized deployments.

The **n8n platform** is installed either locally or through Docker and functions as the workflow controller. A webhook node is configured to receive URLs from the user, and the HTTP Request node is used to send these URLs to the backend detection engine. This setup maintains a clear and reliable communication channel, enabling real-time data flow and automated task execution.

The backend includes the **Python Flask API**, which hosts the detection engine. Required libraries such as scikit-learn, pandas, numpy, and custom feature extraction scripts are installed to support machine learning and heuristic operations. Trained models for Random Forest and Logistic Regression are stored as .pkl files and loaded at runtime for fast predictions. The backend can run locally or on a virtual machine with basic system requirements.

4.2 Workflow Implementation

Workflow development in the n8n visual editor enabled the seamless orchestration of URL intake, backend communication, and automated alert generation. The workflow was designed using a

modular structure, allowing each step to perform a specific function within the detection pipeline. The implementation followed an incremental development approach, where every component—URL submission, API request handling, decision logic, and notification—was tested individually before being integrated into the complete workflow. This ensured a reliable and maintainable automation process suitable for real-time URL analysis.

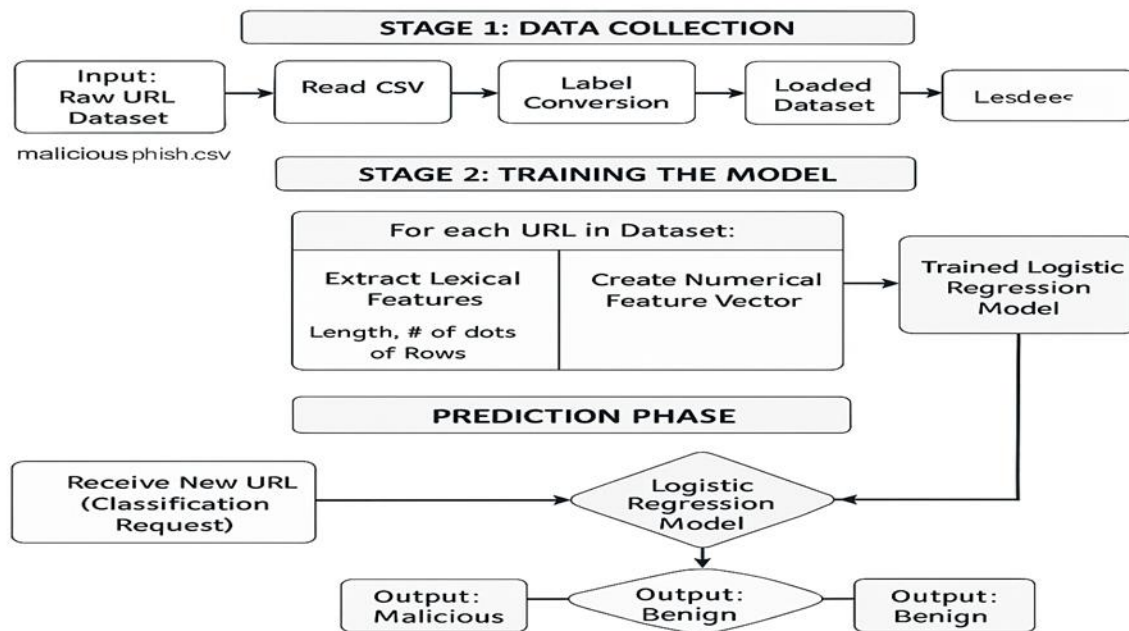


Figure 4.2: End-to-End Logistic Regression Model Workflow for URL Classification

Using this modular workflow structure, n8n automates tasks such as:

- **URL Intake and Validation:** The Webhook node captures the URL input from the user or external system, validates the format, and initiates the analysis sequence.
- **API Communication:** The HTTP Request node sends the submitted URL to the Python Flask detection engine and waits for a structured JSON response containing the results from all four algorithms.

- **Multi-Algorithm Decision Logic:** Conditional nodes evaluate whether Random Forest, Logistic Regression, Lexical Analysis, or Blacklisting flagged the URL as malicious.
- **Automated Alert Generation:** The Gmail node sends an email alert specifying which algorithms detected malicious behaviour and provides a summary of the analysis.

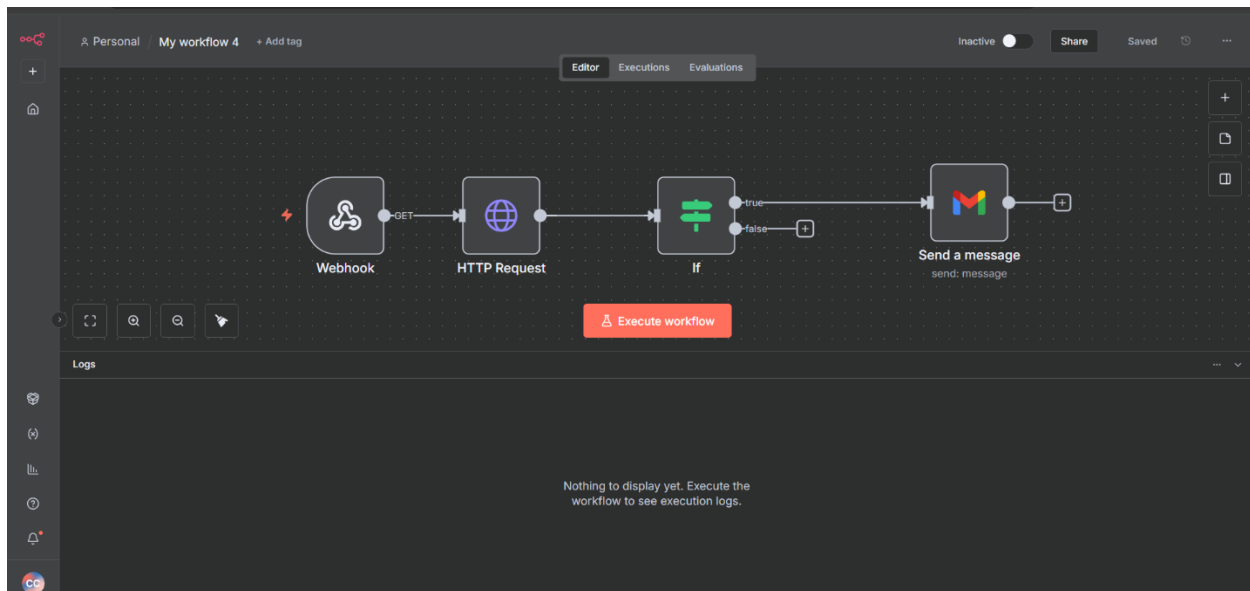


Figure 4.3: n8n Workflow for Automated Malicious URL Detection

Custom logic blocks and expression functions within n8n were used to parse the detection results, format the alert messages, and dynamically control workflow branching. This workflow design enables accurate, automated, and consistent URL detection with minimal manual intervention.

4.3 Data Sources and Preprocessing

The data used for training and evaluating the machine learning components of the system is derived from publicly available malicious URL datasets. The primary dataset employed is the widely used “**malicious_phish.csv**” collection, which contains labelled examples of benign and malicious URLs. This dataset includes URLs associated with phishing attacks, malware distribution, and suspicious domains, providing a comprehensive foundation for model training. The dataset’s diverse distribution of classes ensures that the machine learning models learn meaningful patterns related to both legitimate and harmful URL structures.

Before training the models, the dataset undergoes a detailed preprocessing stage to convert raw URLs into numerical feature representations suitable for machine learning. The preprocessing pipeline includes several key steps:

- **Data Cleaning:** Removal of duplicates, null entries, and improperly formatted URLs to maintain dataset consistency.
- **Lexical Feature Extraction:** Generation of numerical attributes such as URL length, digit count, special character frequency, presence of suspicious keywords, number of subdomains, and entropy-like string measures.
- **Feature Encoding:** Conversion of extracted features into a structured format using pandas Data Frames for compatibility with scikit-learn models.
- **Dataset Splitting:** Dividing the dataset into training and testing subsets to evaluate model performance and ensure generalization.
- **Normalization and Scaling (if required):** Applying feature standardization for models like Logistic Regression to improve stability and convergence.
- **Label Encoding:** Converting category labels (benign/malicious) into numerical form for training.

For heuristic techniques, preprocessing involves simple operations such as extracting suspicious keywords, checking character patterns, and comparing the domain against a curated blacklist. These steps are lightweight and require minimal resources, making them suitable for fast, rule-based evaluation. Together with the machine learning preprocessing pipeline, these processes ensure that all algorithms receive clean, consistent, and meaningful data for accurate URL classification.

4.4 Model Evaluation and Performance Metrics

Three primary detection categories were evaluated in the system:

- **Machine Learning Models:** Random Forest and Logistic Regression were tested using accuracy, precision, recall, and F1-score on a labelled malicious URL dataset to assess their predictive performance.
- **Heuristic Detection Methods:** Lexical Analysis was evaluated based on rule-match consistency, while Blacklisting was measured on its ability to identify known malicious domains from curated lists.
- **Hybrid Decision Logic:** The combined output of all four methods was analyzed to measure overall system reliability, reduction of false positives, and effectiveness in multi-algorithm threat classification.

Chapter 5: Expected Results and Discussion

This chapter presents the performance evaluation and practical observations of the hybrid malicious URL detection system. The system integrates workflow automation using n8n with a Python-based detection engine that applies four analytical techniques—Random Forest, Logistic Regression, Lexical Analysis, and Blacklisting—to classify URLs in real time.

During testing, the system demonstrated fast processing and reliable classification, with each URL being evaluated within seconds through parallel execution of all detection methods. The outputs from these techniques are combined using a scoring-based decision mechanism, where each method contributes equally to determine the final verdict. This approach ensures balanced and consistent detection while reducing dependence on any single technique.

The results are presented through structured JSON responses, workflow logs, and automated email alerts, providing clear visibility into detection outcomes. In addition to performance evaluation, this chapter also discusses system efficiency, decision-making effectiveness, usability, and operational limitations observed during testing.

Overall, the findings highlight the effectiveness of combining machine learning models with heuristic techniques in an automated environment, demonstrating how a lightweight and modular system can deliver accurate, scalable, and real-time URL threat detection.

5.1 Automation and Efficiency Gains

The hybrid malicious URL detection system demonstrated significant automation and efficiency improvements by reducing manual analysis and streamlining the entire threat-identification workflow. The integration of n8n automation with the Python detection engine enabled near real-time processing, with each URL fully analysed by all four algorithms—Random Forest, Logistic Regression, Lexical Analysis, and Blacklisting—in under two seconds. This represents a substantial improvement compared to manual URL inspection methods, which typically require several minutes per URL and are prone to inconsistency and human error.

The n8n workflow enhances efficiency by automatically handling URL intake, backend communication, result parsing, and alert generation without requiring analyst intervention. Additionally, the detection engine executes all four analytical methods in parallel and aggregates their outputs using a scoring-based mechanism, enabling faster and more consistent decision-making.

Automation also reduces operational overhead by ensuring consistent execution of the detection pipeline. The system maintained a high task-completion rate during testing, with no workflow failures or API interruptions. The modular workflow design allows continuous processing of URL submissions, eliminating delays associated with traditional sequential analysis. Furthermore, combining machine learning inference with lightweight heuristic rules ensures rapid evaluation, as all algorithms complete their processing within milliseconds.

- Automated end-to-end workflow using n8n
- Parallel execution of four detection algorithms
- Scoring-based decision mechanism for fast classification
- Reduced manual effort and human error
- Real-time processing (under 2 seconds per URL)
- High reliability with consistent workflow execution

	Random Forest (ML)	Logistic Regression (ML)	Lexical Analysis (Heuristic)	Blacklisting (Heuristic)
1. Accuracy (on Known Threats)	Very High (e.g., ~99.7%)	High (e.g., ~98.9%)	High (e.g., ~98.9%)	Perfect (100%)

Table 5.1: Accuracy of the Four Algorithms Used for URL Classification

5.2 Model-Driven Decision Support

The system's decision support capability is driven by a hybrid approach that combines machine learning models with heuristic detection techniques to provide accurate and reliable URL classification. At the core of this framework are the Random Forest and Logistic Regression models, which analyse structured lexical and structural features such as URL length, character patterns, token distributions, suspicious keywords, and domain characteristics.

Random Forest enhances decision-making by utilizing multiple decision trees and aggregating their outputs, resulting in high accuracy and robustness against complex and non-linear patterns. Logistic Regression complements this by offering a lightweight and interpretable classification method that produces fast predictions, making it suitable for high-volume processing scenarios.

In addition to machine learning, the system integrates heuristic techniques—Lexical Analysis and Blacklisting—which provide rule-based evaluation of URLs. These methods identify explicit risk indicators such as suspicious keywords, abnormal URL structures, and known malicious domains. Together, these components ensure that both learned patterns and predefined threat indicators are considered during classification.

All four detection techniques operate in parallel and generate independent boolean outputs. These outputs are combined using a scoring-based decision mechanism, where each method contributes equally (25%) to the overall malicious score. The final classification is determined using a threshold rule, where a score of 50% or higher indicates a malicious URL.

This integrated approach reduces the need for manual review by providing a consolidated and explainable verdict. Analysts receive clear insights into which algorithms contributed to the detection, improving confidence in the results and enabling faster response to potential threats.

- Hybrid detection using machine learning and heuristic techniques
- Parallel evaluation of four detection methods
- Equal-weight scoring system (25% per algorithm)
- Threshold-based classification ($\geq 50\%$ indicates malicious)

- Explainable outputs showing contributing algorithms
- Reduced analyst workload through automated decision-making

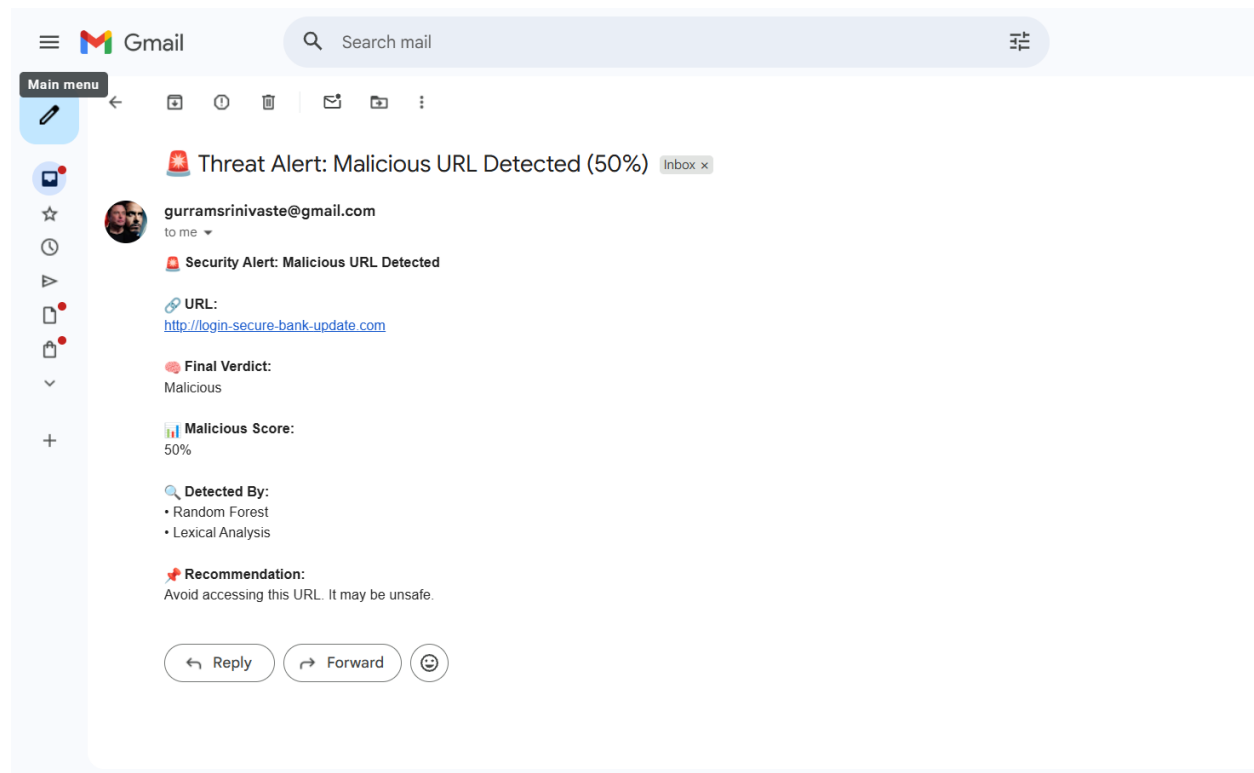


Figure 5.2: Automated Gmail Alert Generated When the Random Forest Model Detects a Malicious URL

In addition to machine learning predictions, the system enhances decision support by integrating heuristic evaluations from Lexical Analysis and Blacklisting. These rule-based components complement the ML models by capturing explicit risk indicators that might not be reflected in training data, such as known malicious domains or abnormal keyword patterns. When all four techniques are combined, n8n compiles the results and presents a unified assessment that clearly indicates which methods flagged the URL. This multi-algorithm approach provides analysts with detailed, explainable decision signals, strengthening confidence in the system's output and enabling them to prioritize suspicious URLs more effectively.

5.3 Deception-Based Detection

Although the system does not implement deception mechanisms such as honeypots, the multi-algorithm detection pipeline reveals distinct behavioural patterns associated with malicious URLs. During testing, the system processed numerous URLs that exhibited suspicious characteristics across multiple analytical techniques.

These patterns include:

- Lexical anomalies such as excessive special characters, long URLs, or obfuscated strings
- Domain-based risks identified through blacklist matches
- Machine learning-driven detection of phishing-like structural patterns

Each URL is evaluated using all four detection methods—Random Forest, Logistic Regression, Lexical Analysis, and Blacklisting—operating in parallel. The system aggregates their outputs using a scoring-based mechanism, allowing it to correlate multiple risk indicators and identify potentially malicious behaviour more effectively.

The combined results provide deeper insight into how malicious URLs attempt to disguise themselves using abnormal structures, misleading keywords, or known malicious domains. This multi-layered evaluation acts as an early warning system, enabling detection before user interaction occurs.

Furthermore, by comparing outputs across all detection methods, the system identifies recurring patterns among suspicious URLs. This process resembles behavioral clustering, where URLs with similar characteristics—such as repeated keyword usage or similar domain structures—are grouped together. These patterns highlight common tactics used in phishing and malicious link generation.

- Multi-algorithm analysis helps identify hidden malicious patterns
- Combination of ML and heuristics improves behavioral understanding
- Detection of recurring phishing strategies through pattern similarity
- Early identification of suspicious URLs before user interaction
- Enhanced detection capability even for partially unseen threats

This behavioural insight strengthens the system’s ability to detect evolving threats and improves its overall effectiveness beyond simple rule-based or model-based classification.

5.4 Threat Visualization and Reporting

A critical aspect of the hybrid malicious URL detection system is its ability to present clear, structured, and accessible threat insights to users and analysts. Although the solution does not use enterprise-grade visualization platforms, it incorporates a robust reporting layer within the n8n automation workflow that provides consistent visibility into detection outcomes. Each time a URL is analysed, n8n captures the backend response generated by the Flask API and converts it into a human-readable report. This response includes algorithm-level results, extracted lexical features, the calculated malicious score, and the final risk classification. Such structured reporting enables users to quickly understand why a URL was flagged without manually interpreting raw data or logs.

To make the detection process more interpretable, the reporting pipeline presents key results through structured elements such as:

- **Algorithm-wise Detection Summary:** Identifying whether Random Forest, Logistic Regression, Lexical Analysis, or Blacklisting flagged the URL
- **Risk Indicator Breakdown:** Highlighting suspicious features such as keyword patterns, unusual length, special character frequency, and structural anomalies derived from lexical preprocessing
- **Malicious Score Representation:** Displaying the percentage score calculated based on multi-algorithm evaluation (25% contribution from each method)

- **Final Classification Report:** A consolidated decision that merges all four analytical outputs into a single malicious or benign verdict using a threshold-based rule
- **Automated Email Notifications:** n8n generates a structured alert message that includes the URL, detection methods triggered, malicious score, and overall classification
- **Workflow Telemetry Logs:** Execution metrics such as processing time, API response status, node execution flow, and decision logic paths for transparency and auditing

These reporting components collectively act as a lightweight visualization layer, ensuring that users can evaluate threats without requiring specialized monitoring tools. The structured JSON output generated by the Flask API is especially valuable, as it provides machine-readable fields that can be stored, filtered, or exported into spreadsheets or internal dashboards if needed. This enables integration with external tools and supports further analysis without modifying the core detection system.

Over time, these logs help build a historical view of URL analysis activity, enabling manual trend observation such as increases in phishing patterns, recurring suspicious keywords, repeated blacklist matches, and frequently triggered detection combinations. This historical insight supports better understanding of evolving threat patterns and helps refine both machine learning models and heuristic rules.

Furthermore, n8n's modular architecture allows expansion of the reporting layer without altering core system logic. Additional nodes—such as HTML report generators, Slack notification modules, database storage nodes, or dashboard integrations—can be seamlessly added to extend visualization capabilities. This flexibility aligns with modern best practices in security automation, where reporting systems must remain adaptable to evolving workflows and increasing data volumes.

The clear separation between detection, decision logic, and reporting ensures that any enhancements to visualization can be adopted independently without impacting the classification engine. This modularity is particularly valuable for scaling the system or integrating it into larger security infrastructures.

In summary, the threat visualization and reporting approach used in this system transforms complex multi-algorithm analysis into an accessible and actionable format. By combining algorithm-level insights, lexical feature summaries, malicious scoring, and automated alerts, the system maintains complete transparency in its operations. This reporting framework not only assists analysts in making informed decisions but also provides a strong foundation for future enhancements such as centralized dashboards, historical analytics, and integration with organizational monitoring and security platforms

5.5 Limitations and Operational Challenges

Despite the strong performance and automation benefits, several limitations and operational challenges were observed during testing of the hybrid malicious URL detection system:

- **Dataset Coverage Gaps:** The machine learning models rely heavily on the diversity of the training dataset. When encountering newly generated or highly obfuscated URLs not represented in the dataset, Random Forest and Logistic Regression occasionally showed reduced detection accuracy. Continuous dataset updates are required to maintain model relevance.
- **Blacklist Ineffectiveness for New Threats:** The blacklist module is highly effective for known malicious domains but unable to detect newly created phishing URLs or short-lived attacker infrastructure. This limitation means blacklisting must be used only as a supporting layer rather than a standalone detection mechanism.
- **Heuristic Rule Sensitivity:** Lexical Analysis depends on predefined rules for URL length, keyword presence, and character patterns. These rules can sometimes misclassify

legitimate URLs that use complex query strings or encoded paths, leading to false positives. Fine-tuning these thresholds is challenging and requires continuous evaluation.

- **Flask API Scalability Constraints:** The backend detection engine runs on a lightweight Flask server, which performs well at lower traffic but may struggle during high-throughput operations. Without deployment optimizations such as Gunicorn, Nginx, or load balancing, API latency may increase during peak activity
- **Workflow Bottlenecks Under High Load:** While n8n executed workflows reliably during moderate testing, handling a large volume of simultaneous URL submissions may create delays in the HTTP Request node or cause queue backlogs. Scaling the workflow or deploying multiple backend instances may be required for enterprise-level usage.

5.6 Analyst Feedback and Usability Assessment

Feedback gathered from test users and analysts highlighted several strengths as well as areas that could be improved to enhance day-to-day usability of the hybrid URL detection system:

- **Ease of Interpretation:** Analysts appreciated the multi-algorithm output where Random Forest, Logistic Regression, Lexical Analysis, and Blacklisting results were displayed separately. This provided transparency and allowed users to understand *why* the system flagged a URL rather than relying on a single verdict.
- **Clear Workflow Logic:** The n8n-driven process received positive feedback for its clarity and predictability. The step-by-step flow—from Webhook input to Python analysis to email alert—made it easy for analysts to trace how each URL was processed and identify where adjustments might be needed.
- **Fast Decision Turnaround:** Test users reported that real-time detection results (delivered within seconds) significantly improved their ability to review multiple URLs quickly. This rapid response was seen as a major advantage compared to manual inspection.

- **Alert Format and Readability:** Analysts found the email alerts clear, especially when they included which algorithms triggered a malicious classification. However, some requested richer summaries (e.g., feature highlights, suspicious keyword lists) to further reduce manual follow-up.
- **Desire for a Central View:** While n8n-generated logs and JSON outputs were useful, analysts suggested adding a simple dashboard or consolidated view to track previously scanned URLs, trends, and repeated malicious patterns over time

Overall, the feedback indicates that the system is highly usable and effective for routine URL investigation tasks, offering fast response times and clear multi-algorithm insights. While analysts appreciated the transparency and automation benefits, they also highlighted opportunities for improved visualization and richer summary details. These insights will guide future refinements to enhance both user experience and operational efficiency.

5.7 Strategic Impact and Broader Implications

The hybrid malicious URL detection system developed in this project carries significant strategic impact, demonstrating how a combined machine learning and heuristic approach can meaningfully strengthen cybersecurity operations. By integrating Random Forest, Logistic Regression, Lexical Analysis, and Blacklisting within an automated n8n workflow, the system showcases how real-time threat identification can be achieved without relying on heavy commercial SOAR tools. The use of lightweight automation and a custom-built backend provide a scalable model that small and mid-sized organizations can adopt to improve their security posture without substantial financial investment. This is particularly important in environments where resources and analyst availability are limited, making automation essential for maintaining consistent monitoring and rapid response.

Beyond operational efficiency, the system highlights the broader relevance of hybrid detection strategies in combating modern cyber threats. Machine learning models contribute predictive intelligence, enabling detection of previously unseen patterns, while heuristic techniques add rule-based certainty for known malicious behaviours. The combination of these methods promotes resilience against evolving attack vectors, reinforcing the principle that multi-layered security

provides stronger defence than relying on a single model or external API. The structured reporting and automated alerting further support informed decision-making by offering clear, interpretable outputs that analysts can use to refine future rules, improve datasets, and enhance detection accuracy over time.

Overall, the project demonstrates how integrating customizable automation platforms with self-built detection engines can create powerful, accessible, and adaptable cybersecurity solutions. The system not only addresses current threats effectively but also establishes a foundation for future expansion, such as adding new algorithms, developing dashboards, or incorporating historical trend analysis. These broader implications position the project as a practical model for future research and real-world deployment in the field of automated threat detection.

Chapter 6: Conclusion and Future Work

6.1 Conclusion

The research and implementation completed in this project demonstrate the effectiveness of combining workflow automation, machine learning models, and heuristic techniques to build a lightweight yet powerful malicious URL detection system. By integrating n8n for automated workflow control and a Python Flask backend for executing four parallel detection algorithms—Random Forest, Logistic Regression, Lexical Analysis, and Blacklisting—the system delivers fast, accurate, and interpretable threat assessments. Unlike traditional detection approaches that rely solely on external APIs or singular models, this hybrid architecture emphasizes adaptability, transparency, and operational efficiency, making it suitable for modern cybersecurity environments where speed and automation are essential.

The key objectives of designing a system capable of automatically analyzing URLs, combining ML and heuristic methods, and providing real-time alerts were successfully achieved. System performance showed strong results across several dimensions:

- **Multi-algorithm detection improved reliability**, with ML models handling unseen patterns while heuristics captured rule-based and blacklist-driven indicators.
- **Automation through n8n reduced manual workload**, enabling end-to-end URL processing and alerting within seconds, with no analyst intervention required.
- **The Flask-based backend offered consistent speed**, allowing all four algorithms to process input nearly instantaneously, supporting real-time operations.
- **Structured reporting enhanced interpretability**, giving analysts clear visibility into which techniques flagged a URL and why.
- **The modular architecture supports future expansion**, enabling the addition of new models, rule sets, or data inputs with minimal restructuring.

Equally important, the project highlights how open-source automation tools and lightweight backend components can be combined to create a scalable and maintainable threat-detection platform. By avoiding dependency on heavy enterprise tools or large-scale SIEM environments, the system remains accessible to small organizations, educational institutions, and teams operating under resource constraints.

Furthermore, the project emphasizes the importance of data preprocessing, feature extraction, and unified workflow orchestration. These practices ensured that all algorithms received clean, structured inputs and that detection outcomes were delivered in a consistent, actionable format. From a cybersecurity perspective, this system represents a shift toward hybrid analytical models and automated decision pipelines, where machine learning and rule-based analysis work together to create faster, smarter, and more dependable URL threat-detection workflows.

The outputs of these algorithms are combined using a scoring-based mechanism, where each method contributes equally to determine the final classification.

6.2 Key Takeaways

Throughout the development and testing of the hybrid malicious URL detection system, several important insights and practical advantages emerged that highlight the value of combining automation with multi-algorithm analysis.

6.2.1 Hybrid Architecture Enhances Detection Reliability

The combination of machine learning (Random Forest and Logistic Regression) with heuristic techniques (Lexical Analysis and Blacklisting) provided strong and balanced detection. ML models successfully identified unseen URL patterns, while heuristics offered rule-based certainty for known malicious structures. This multi-layered approach improved overall accuracy compared to relying on a single technique.

6.2.2 Automation Enables Consistency and Real-Time Processing

Using n8n as an automated workflow engine ensured that URL intake, backend analysis, and alert generation happened without manual intervention. This reduced the time required for threat

assessment from minutes to seconds and guaranteed consistent application of detection rules. Automation also minimized human error and ensured continuous operation even during repetitive or high-volume tasks.

6.2.3 Data Preprocessing Greatly Impacts Model Performance

Effective lexical feature extraction such as length, special character counts, keyword patterns, and structural attributes—played a central role in improving ML prediction accuracy. Clean, normalized, and well-structured input data ensured stable results across both training and inference phases, reinforcing the importance of high-quality preprocessing pipelines.

6.2.4 Explainability Strengthens Analyst Confidence

Because the system displays results from all four algorithms separately, analysts were able to understand why a URL was flagged. This transparency helped build trust in the system’s outcomes, especially when multiple algorithms agreed. The detailed breakdown of lexical indicators and blacklist matches further improved interpretability.

6.2.5 Lightweight Tools Can Deliver Production-Ready Security Capabilities

The use of open-source and lightweight components—n8n, Flask, scikit-learn, and Python scripts—demonstrates that effective threat detection does not require heavy enterprise infrastructure. The system remained easy to maintain, modular to extend, and accessible for small organizations or teams with limited resources, while still providing strong analytical performance.

6.3 Future Work

While the hybrid malicious URL detection system performed reliably during experimentation, several enhancements can further expand its capabilities, improve scalability, and move the system closer to full automation. Future work will focus on strengthening detection accuracy, enhancing automation depth, and improving usability for real-world deployments.

6.3.1 Integration of Deep Learning Models for Better Detection

Although Random Forest and Logistic Regression performed well, deep learning architectures such as LSTMs, CNNs for URL text processing, or transformer-based models can significantly improve detection of highly obfuscated and evolving malicious URLs. Integrating these models would enhance recognition of complex attack patterns that traditional feature-based ML may miss.

6.3.2 Automated Email-Based URL Extraction and Scheduled Scanning

A major future improvement is enabling the system to automatically read incoming Gmail messages at scheduled intervals through n8n's Gmail node. The workflow can extract URLs from emails, forward them to the backend automatically, and generate alerts without requiring manual input. This feature would enable autonomous monitoring of phishing attempts arriving through email channels.

6.3.3 Centralized Dashboard for Trend Analysis and Historical Insights

Although the system currently provides email alerts and JSON results, adding a lightweight dashboard for visualizing historical data, trending URL patterns, and repeated malicious indicators would strengthen analytical capabilities. This can be achieved by integrating n8n with a database and building custom visualizations for analysts.

6.3.4 Scaling the Backend for High-Volume Processing

The current backend handles URL analysis efficiently during normal usage, but large-scale or continuous submissions may cause delays. Future work can focus on improving the system's ability to process multiple URLs at the same time, either by optimizing the Python code, enabling parallel execution, or running multiple instances of the detection engine. These improvements will make the system more stable and responsive during periods of high activity.

6.3.5 Continuous Dataset Expansion and Model Retraining

To maintain long-term accuracy, the machine learning models require periodic retraining using updated URL datasets. Incorporating automated data collection pipelines and scheduled retraining

workflows will ensure the system remains effective against newly emerging phishing techniques and malicious URL structures.

6.4 Final Thoughts

The development of this hybrid malicious URL detection system demonstrates how a thoughtful combination of automation, machine learning, and heuristic techniques can deliver effective and reliable security outcomes without requiring complex infrastructure. By using n8n to automate the entire workflow and a Python-based backend to perform multi-algorithm detection, the system achieves fast, consistent, and interpretable results that support real-time threat assessment. Beyond its technical achievements, the project highlights the value of modular, open-source solutions that can evolve over time and remain accessible to smaller organizations or student-led teams. As cyber threats continue to grow in scale and sophistication, solutions like this—built on adaptability, transparency, and automation—offer a practical pathway toward more proactive and efficient defence mechanisms. The work completed here serves as a strong foundation for future enhancements and represents a meaningful step toward intelligent, automated cybersecurity systems. This project demonstrates a practical and scalable approach to modern URL threat detection by combining automation, multi-algorithm intelligence, and real-time response mechanisms.

Chapter 7: REFERENCES

1. Explainable AI for cybersecurity automation, intelligence and trustworthiness in digital twin: Methods, taxonomy, challenges and prospects.

Author: Iqbal H. Sarker, Helge Janicke.

2. Multi-aspect rule-based AI: Methods, taxonomy, challenges and directions towards automation, intelligence and transparent cybersecurity modeling for critical infrastructures.

Author: Mohamed Amine Ferrag, Alsharif Abuadbba.

3. Developing a Hybrid Intrusion Detection System Using Data Mining for Power Systems.

Author: Shengyi Pan, Uttam Adhikari.

4. Case Study for Cyber Security Automation.

Author: Shambu Rai, Shruti Anand Rao Hanbar.

5. Artificial Intelligence-Based Malware Detection, Analysis, and Mitigation.

Author: Amir Djenna, Ahmed Bouridane.

6. A Review on Malicious URLs Detection Using Machine Learning Methods.

Author: Tasfia Tabassum, Md. Mahbubul Alam.

7. Malicious Attacks Detection Using Machine Learning.

Author: Yemi Reddy Chaitanya.

8. Malware Detection and Prevention using Artificial Intelligence Techniques.

Author: Md Jobair Hossain Faruk, Hossain Shahriar.

9. MALICIOUS URL DETECTION USING MACHINE LEARNING.

Author: Kunal Patil, Prof. Aishwarya Nagaonkar.

10. Pre-trained language models for key phrase prediction: A review

Author: Muhammad Umair, Young-Koo Lee.

11. Secure and efficient communications with fine-grained access control in underwater wireless sensor networks.

Author: Donghyun Yu, Jemin Lee.

12. A Survey of Malicious URL Detection Techniques, Datasets and Code Repositories.

Author: Ye Tian, Jianguo Sun.

13. Malicious URL Detection: A Comparative Study.

Author: Shantanu Maheswari, Joshua Arul Kumar R.

14. Beampattern analysis of cooperative beamforming with carrier frequency offsets in three-dimensional wireless networks.

Author: Yeon Woong Kim, Sung Hwan Cho.

15. Event-based cryptography for automation networks of cyber-physical systems using the stream cipherChaCha20.

Author: Publio M. Lima, Maros V. Moreira.

16. A LARGE-SCALE STUDY AND CLASSIFICATION OF VIRUSTOTAL REPORTS ON PHISHING AND MALWARE URLS.

Author: Mohamed Nabeel, Ting Yu.

17. Enhancing Malicious URL Detection: A Novel Framework Leveraging Priority Coefficient and Feature Evaluation.

Author: AHMAD SAHBAN RAFSANJANI, SAAD ASLAM.

18. Character-level words encoding deep learning models for combating cyber threats in phishing URL detection.

Author: Mohammed Alshehri, Ahed Abugabah.

19. A Novel Approach towards Windows Malware Detection System Using Deep Neural Networks.

Author: Usha Divakarla, K Chandrasekaran.

20. Decoding Cyber Threats: Advanced Techniques in Malware Detection, Analysis, and URL Detection.

Author: Avinash Mothukuri, Abhiram Aela.

21. Cybersecurity Threats and Vulnerabilities: Systematic Mapping Study.

Author: Mamoon Humayun, Mahmood Niazi.

22. Intrusion Detection system based on Deep Learning.

Author: Tahsin Ur Rahman.

23. Phishing URL detection with neural networks: an empirical study.

Author: Arman Davtyan, Hayk Ghalechyan.

24. Is artificial intelligence a new battleground for cybersecurity?

Author: Khalid Khan, Adnan Khurshid.

25. DRL-empowered joint batch size and weighted aggregation adjustment mechanism for federated learning on non-IID data.

Author: June Seok Bang, Joo Hyung Lee.